

Evidence-Bound Recovery for Ambiguous Submissions in Local Generative-AI Pipelines

Tool Description

Anonymous submission draft

Abstract—A lost submission response can leave a generation client uncertain whether its backend executed a request. We describe evidence-bound recovery tooling for local image-generation pipelines: an evidence exporter, a same-run submission guard, and controlled fault injection with backend observation. The exporter separates reported state, interpretation, and oracle evidence. Four fixed Windows/ComfyUI cases retained one execution binding per control and two per response-loss case. Each response-loss case had an unresolved first call and a resolved second call. The second call created a fresh run outside the guard’s scope; this demonstration does not establish guard efficacy or recovery of the original run. Only synthetic CPU fixtures exercise the guard, exposing incomplete operation as a cost of withholding submission. Retained bindings support record-level auditing, not independent reconstruction of all backend events. Deployment rates and output-quality improvements remain unmeasured.

I. INTRODUCTION

Consider a client that submits an image-generation workflow and loses the response after the backend accepts it. The backend may continue while the client lacks the returned job identifier. A second submission creates another opportunity for execution, but a timeout or a count of HTTP requests cannot establish what the backend actually did. The relevant question is which observations justify the next recovery action and the subsequent report of that action.

Uncertain outcomes are an established distributed-systems problem. Remote procedure call designs distinguish call semantics under communication failure [1], while idempotent API designs use caller-provided identifiers and explicit service behavior to make retries safe [2]. A local workstation does not remove the distinction between request transmission and backend execution. Our tool addresses how that distinction is represented and inspected in a particular local generation pipeline; it does not introduce a new exactly-once protocol.

Our prototype combines persistent run records, generation orchestration, a ComfyUI adapter and artifact handling. We describe tooling that preserves the client’s evidence, applies versioned interpretation rules, and records backend observations independently of the client’s returned status. Its product guard is deliberately narrower: an explicitly unknown submission blocks further submission through the same unresolved run at the reviewed entry points.

The tool description has three parts: an evidence representation and read-only exporter; a same-run guard whose unresolved state remains visible; and CPU and Windows fault-observation workflows with separate evidence boundaries.

The Windows demonstration tests visibility across the client-backend boundary. It does not establish that the guard reduces execution counts in that protocol. The contribution is an inspectable integration of evidence interpretation, run-scoped control and backend observation. Section VI compares its scope with idempotent APIs, workflow execution and tracing. The evaluated contribution is this implementation and its failure windows.

II. FAILURE MODEL AND EVIDENCE SEMANTICS

A **product call** is an invocation by the research client. A **run** is the persistent product object created by `local_gpu_start_run`. A **submission** in the Windows results is a proxy-observed `POST /prompt`; a call that fails before that boundary need not produce a submission. An **execution instance** is an observation bound to backend execution events. CPU fixtures instead observe the synthetic worker entry. These units are not interchangeable.

F00 is a normal control. F02 suppresses the first accepted submission response, creating a window in which the client lacks an acknowledged job identifier. F03, covered by the CPU fixtures rather than the Windows campaign reported here, concerns loss of completion information after a job identifier is known. An explicit rejection and a missing response have different evidential meanings. Neither should be silently converted into a backend execution outcome.

The evidence model distinguishes reported state, interpreted state and oracle state. In addition, the Windows research client emits its own call-level labels. Its `resolved` label is derived from a generated product result after checking for artifact hashes. Its `unresolved` label can arise from several client errors. Thus a research-client label alone does not prove that a durable product manifest has a particular recovery state. In F02, the proxy’s accepted-response loss and the oracle binding provide the additional context.

III. TOOL DESIGN

A. Evidence-preserving export

The exporter accepts a record or run manifest and produces a separate normalized record. It copies reported state, records the mapping version and reasons, and retains supplied oracle state separately. The schema includes source identity, request and input digests, backend identity, job identifier, artifact

hash and validator version. Missing fields carry explicit reasons instead of fabricated values. The inspected mapping is `research-normalization-v2`.

Interpretation checks more than an execution-success label. Job and artifact bindings, approval and validation evidence, and recovery state constrain the `execution_verified` decision. An unresolved recovery state prevents this flag from becoming true even when other evidence reports successful execution. This permits the record to express backend completion without asserting that the product’s recovery obligation has been discharged. The exporter does not invoke the engine, retry a request or update the source run. Its output writer uses exclusive file creation. These are implementation properties, not a claim of complete filesystem security. The interpretation assumes that supplied oracle and artifact-validator records have trustworthy provenance. The exporter checks their stated fields and bindings; it does not authenticate their origin or reconstruct missing raw events. In the current mapping, absent source revision, backend-instance or validator-version metadata need not by themselves prevent `execution_verified`. That flag denotes satisfaction of this mapping’s evidence policy, not independent certification of the input record.

B. Using the evidence record

An investigator first inspects the run manifest and the recorded submission outcome. If submission is explicitly unknown, inspection must preserve that uncertainty: the missing job identifier must not be synthesized from a later successful call. Backend observation is then associated with the relevant request and execution instance, while artifact checks retain their own result. The export command writes a new record and refuses to overwrite an existing export:

```
python -m
scripts.research.export_records
-input run-record.json -output
evidence.json
```

The exported record lets a reader inspect reported state, interpreted state, mapping reasons and oracle evidence without invoking recovery. It does not acquire a missing oracle automatically. The Windows observer export and the normalized exporter schema are separate interfaces; a complete automatic conversion between them is not established by the pilot. The companion offline demonstration supplies two synthetic records that differ in the original run’s status. It runs the exporter and the retained-record audit without a backend. These inputs illustrate the mapping; their supplied oracle assertions are not experimental observations or authenticated evidence.

For example, an unresolved run and successful backend evidence may coexist. The execution field can describe the observed backend outcome while recovery remains required and execution verification remains false. This separation is the intended diagnostic output, rather than an error to hide by replacing the original run state with the latest call’s result.

C. Recovery guard and its scope

`AssetRunEngine` and `RunStore` preserve the explicit transport marker `submission_outcome=unknown` as an unresolved attempt. The reviewed submission entry checks that unresolved attempt before permitting further generation. The guard applies within the same run, including attempts with a different idempotency key. The corresponding inspection action is `get_run`.

This guard does not discover a missing backend job identifier or reconcile an unknown job automatically. A newly created run lies outside the prior run’s guard. Existing recovery for a known job in a two-stage operation is a separate path and is not attributed to this no-job-ID guard. The resulting tradeoff is explicit: the guard withholds another submission through that run while the required evidence is unavailable; it does not guarantee eventual completion.

A pipeline integrator can use this distinction when a generation UI receives a disconnect after sending a request: the exporter preserves the unknown outcome, the same-run entry blocks another submission, and the operator must inspect or reconcile the run through an available backend-specific mechanism. The guard can also conservatively withhold work when a request failed before backend acceptance, so its false-positive cost is delayed completion rather than a claimed reliability gain.

D. Controlled observation on Windows

Figure 1 shows the product and research boundaries. The research harness adds a transport shim and loopback fault proxy. The shim routes prompt transport through the proxy while retaining the actual backend endpoint in the product’s identity client. F02 drops the first response only after backend acceptance. This arrangement is a research instrument, not a native ComfyUI recovery feature.

The observer connects before the product invocation and uses the operation key as its ComfyUI client identifier, matching the product’s event-routing identity. `ComfyUIEventOracle` requires an execution-start event, a terminal event and completed history bound to a prompt identifier. It derives an execution-instance identifier using the backend boot identity, prompt identifier and observed start sequence. Its independence is from the product return value: it still trusts the backend’s event and history interfaces, and it is not a physical GPU monitor or a crash-durable audit log.

The controller permits the second F02 product call only after the first execution has a bindable completion. The client creates a new run on each call and uses `seed = 4100 + call_index`. This is observation-gated fresh-run generation, not automatic reconciliation of the original run. Because the new run is outside the same-run guard, the Windows campaign does not measure the guard’s blocking effect. CPU F02 fixtures exercise blocking; in the single-stage fixture, W3 has one execution rather than B2’s two, while leaving the operation incomplete. Matching frozen request digests does not establish full prompt equality: the inspected digest function excludes the seed and does not use its plan argument.

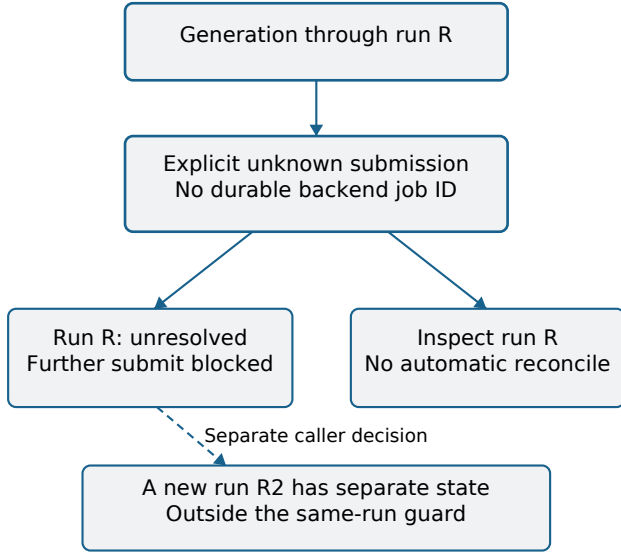


Fig. 1. Scope of the product guard, derived from the reviewed store and engine entry points. A new run has separate state. Dashed arrows denote a separate caller decision, not automatic reconciliation.

IV. EVIDENCE AND DEMONSTRATION

A. CPU components and deterministic fixtures

We denote the frozen baseline product revision by B2 and the revision with the same-run ambiguity guard by W3. The CPU harness separates submission events from synthetic worker-entry events. Its fixtures cover normal operation and response loss before and after job-ID acquisition, including single-stage and two-stage paths. Oracle self-checks exercise cases in which submission and execution counts differ and in which a shared prompt identifier does not imply a single execution. These are fixed semantic checks, not randomly sampled deployments.

Historical command output records 40 research component tests passing in 7.110 seconds using Python unittest discovery, with OK and exit code 0. The preparation report identifies the runtime as Windows Python 3.15.0a8. A later macOS check targeted the transport shim and campaign controller: seven tests passed in 0.003 seconds, followed by successful compilation and whitespace checks. These are distinct stages; the earlier 40-test result is not a full-suite result for the final shim. Neither count is an experimental sample size. Exact immutable source binding of the historical test worktrees remains incomplete.

Table I reports the retained paired CPU matrix. Reanalysis checked all twelve synthetic traces: each execution-start identifier was unique within its case and matched a finish identifier, and request-received counts matched the stored submission totals. The control cases are separate from injected cases. The invalid earlier matrix version, whose single-stage control failed because of a harness result-field mismatch, is excluded.

The distinguishing F02 single-stage fixture has a concrete cost: W3 blocks a second product submission. Its synthetic worker records one successful start and finish with an artifact, yet the product operation remains incomplete in the observation window. B2 submits twice and completes in this fixture. The guard’s direct action is withholding a submission; the corresponding difference in synthetic execution counts is not a measured real-backend reduction. The unchanged F03 two-stage row exercises existing known-job recovery. These CPU paths must not be pooled with the Windows fresh-run protocol in Fig. 2.

B. Four controlled Windows cases

The endpoint-preserving campaign report records B2 and W3 under F00 and F02. All four fixed cases were oracle-evaluable; the two F00 cases were controls and the two F02 cases had confirmed response-loss injection. Table II reports the observations without converting these counts into a success probability.

The campaign totals were six calls, six proxy submissions and six retained bound execution instances. The F02 second call used a fresh run and changed seed. The four report records matched the corresponding retained JSONL cases. Their six retained execution bindings had distinct identifiers, completed-history flags and terminal-event labels. In both B2_F02 and W3_F02, the export reports `execution_start=3`, two proxy submissions and two retained bindings. Each F02 case has two observer decisions. The oracle’s `raw_events` field is cumulative, while the export counts the entire event list of each decision. For example, if the first list contains start A and the second contains A and B, the sum is $1 + 2 = 3$ entries for two distinct starts. Each F00 case has only one decision, so this recounting opportunity is absent there. The uniqueness check concerns the two retained binding identifiers per F02 case, not the three aggregate entries. This source-level mechanism makes the reported count compatible with two bindings; it does not reconstruct the historical lists. Because the original snapshot boundaries and payloads are unavailable, an additional unbound start cannot be independently excluded. We report two retained bindings per case, not a complete execution ground truth. The first binding’s completion precedes the second receipt, as required by the controller’s observation gate; these timestamps belong to different calls. No rate is estimated. Here, “oracle-evaluable” refers to the retained binding record, not an independent raw-event audit.

The report identifies an RTX 5070 Ti Laptop GPU and ComfyUI v0.30.0 at a pinned source revision, with model and workflow hashes checked against the campaign configuration. These are recorded environment facts for this campaign. They do not establish behavior across other models, backend versions or machines.

Both systems produced two retained execution bindings in F02. This demonstrates that the research export retained execution identifiers despite the first call’s ambiguous outcome. The source explains how the aggregate count can exceed the binding count, but the retained extract does not

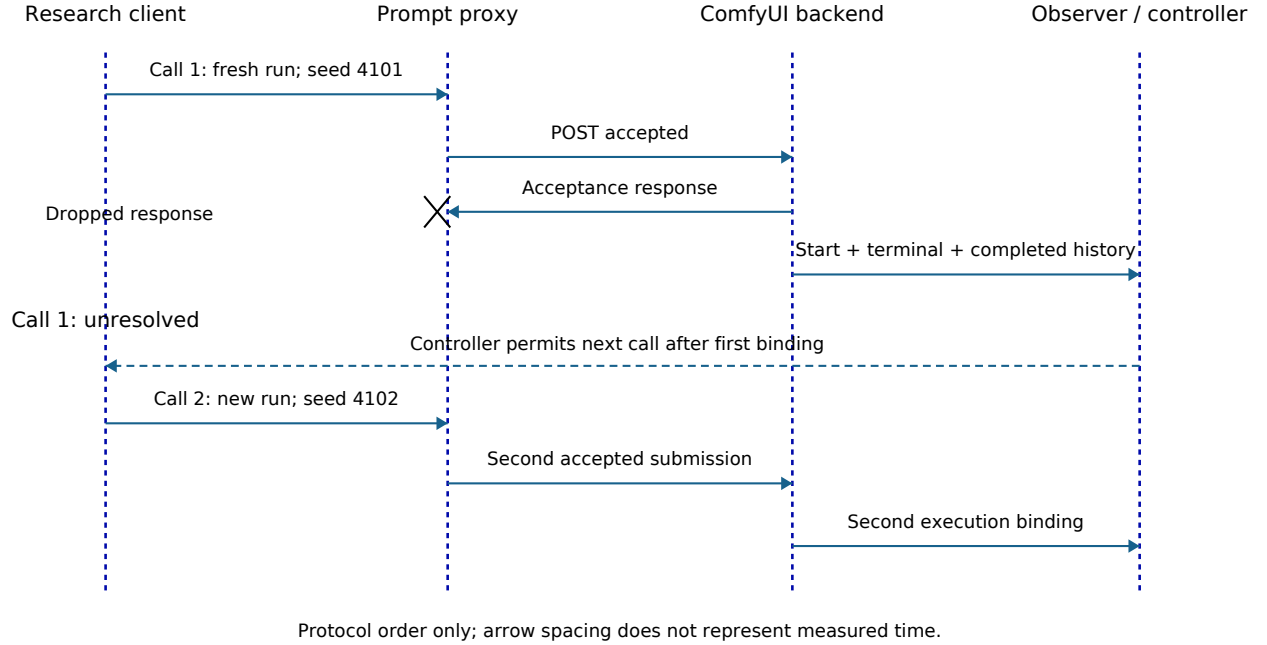


Fig. 2. Controlled F02 protocol. The proxy hides the accepted response; backend observation gates a second call that creates a fresh run with a changed seed. Retained receipts and bindings support the two submissions and the ordering of the second receipt after the first retained binding. The campaign does not exercise the same-run guard. Spacing is schematic, not a measured latency or GPU-runtime plot.

TABLE I
DETERMINISTIC CPU FAULT COVERAGE FROM THE RETAINED PAIRED-V2 MATRIX. S/E MEANS SUBMISSIONS/EXECUTIONS. “COMPLETE” IS THE FIXTURE’S VALID-COMPLETION FLAG. THESE ARE TWELVE SYNTHETIC TRACES, NOT DEPLOYMENT SAMPLES.

Fault	Path	B2 S/E	W3 S/E	Complete (B2/W3)	B2 recovery	W3 recovery
F00	Single stage	1/1	1/1	yes/yes	Not needed	Not needed
F00	Two stage	1/1	1/1	yes/yes	Not needed	Not needed
F02	Single stage	2/2	1/1	yes/no	New submission	Unknown; blocked
F02	Two stage	1/1	1/1	no/no	Unknown	Unknown; blocked
F03	Single stage	2/2	2/2	yes/yes	New submission	New submission
F03	Two stage	1/1	1/1	yes/yes	Same job reconciled	Same job reconciled

TABLE II
FOUR FIXED CASES, CHECKED AGAINST RETAINED REPORT FIELDS. S/E ARE PROXY SUBMISSIONS/BOUND EXECUTION INSTANCES; U/R ARE UNRESOLVED/RESOLVED RESEARCH-CLIENT CALLS. ALL FOUR CASES WERE ORACLE-EVALUABLE.

System	Case	Calls	S/E	Call sequence
B2	F00	1	1/1	R
W3	F00	1	1/1	R
B2	F02	2	2/2	U → R
W3	F02	2	2/2	U → R

independently resolve the historical event sequence. The cases do not demonstrate an execution-count advantage for W3. Because the second call changed run and seed, the table also does not establish a duplicate execution of an identical generation request.

C. Resolution labels require an explicit aggregation rule

A resolved second call does not erase the first call’s unresolved result or show that its original run was reconciled. Read-only inspection of the retained campaign report

confirmed that both F02 records carry `resolved=1` and `unresolved=1`. These are independent any-call flags: across the four cases, `resolved` sums to four and `unresolved` to two. They are overlapping counts, not mutually exclusive outcomes. The earlier Markdown summary’s zero unresolved cases is therefore not the controller aggregation. Table II preserves the call sequence; the evidence addendum records this correction without rewriting the historical report. No recovery-success rate is inferred.

There is also a counting distinction within the observer export. Successive oracle decisions retain cumulative raw-event snapshots, so summing their event counts can count earlier events again. The controller’s execution count uses distinct bound execution-instance identifiers. A raw-evidence audit must check those bindings, not substitute aggregate event counts.

V. REPRODUCIBILITY AND LIMITATIONS

The paired CPU records preserve product revisions, harness digests and event traces. The Windows source and report

identities are recorded separately in the companion evidence ledger. The audit and figure-generation scripts consume existing records only; they do not run the generation backend. Reanalysis is reproducible from the included derived records, while recreating the Windows campaign also requires the pinned backend, model, workflow and private setup. A fresh clean-checkout execution of that campaign was not performed.

The retained Windows JSONL records contain sanitized case summaries, execution bindings, terminal labels and aggregate event counts, rather than full raw WebSocket messages and history snapshots. Thus the audit verifies internal consistency of retained bindings; it cannot independently reconstruct every backend event. The source oracle recognizes completion using an `executing` event with `node=null`, together with completed history. The offline audit’s check of the exported `executing` label is only a consistency check: that label is set by construction and lacks the original node and payload discriminator. The derived, path-redacted evidence extract and a read-only audit script are included with the manuscript source. The offline script checks this derived extract and regenerates the two result tables. The recorded report/JSONL equality and image-byte matches are inspection receipts; rerunning the script does not repeat those host-side checks. No backend call is needed.

Read-only hashing found that the four generated-call artifact hashes matched current output image bytes on the Windows host. Each case had one product-output PNG; the unresolved first F02 call recorded no product artifact hash. Therefore these checks do not verify the first F02 execution’s output bytes. The oracle’s path fingerprints hash metadata strings and cannot substitute for content hashes. Neither a matching content hash nor completed history establishes visual quality or final user acceptance.

Observation depends on receiving and binding backend events within the harness window. Missing events limit evaluability; they do not establish non-execution. The same-process CPU observer and backend-origin Windows observer have different failure boundaries, neither of which establishes crash-complete auditability. The fixed case set contains no basis for deployment failure frequency, natural duplicate-execution frequency, performance improvement or broad model robustness. No GPU run, model download or new experiment was performed for this manuscript revision.

VI. RELATED WORK AND DISCUSSION

RPC failure semantics provide the conceptual starting point: a failed communication does not necessarily settle whether a remote operation ran [1]. Idempotent API design addresses safe retries through an explicit contract, including caller-provided request identifiers [2]. Our tool relies on neither a prompt identifier nor a client idempotency key as sufficient evidence of backend deduplication. Its role is to preserve and qualify observations when such a guarantee has not been established for the evaluated path.

The practical distinction is between observation and control. Backend events can establish that work completed while the

client’s original run remains unresolved. Conversely, a conservative same-run guard can block a further submission without resolving that uncertainty. The Windows harness crosses a different boundary by creating a new run after observation. Keeping these scopes separate explains why the demonstration can be oracle-evaluable without showing automatic recovery or a W3 reduction in executions.

A recent preprint by Li studies ambiguous writes in a simulated service benchmark, comparing model, harness and tool-contract effects on duplicate side effects [3]. Its guard combines read-back verification, idempotency keys where supported, delayed re-verification, blocking unverifiable non-idempotent repeats, and uncertainty annotations. Our guard has a narrower control rule: an explicitly unknown submission blocks further submission through the same run, without automatic read-back or reconciliation. The shared principle of withholding unsafe retries is not a novelty claim. The tool-specific object here is the inspectable integration of preserved reported state, versioned interpretation with reasons, and back-end observation in a local generation pipeline. This is an implementation distinction, not evidence of superiority over that guard or of a new general recovery principle. The studies use different workloads and evidence sources; their outcome counts are not directly comparable.

Reliable State Machines provide a stronger control contract within their programming model: persistent state and runtime-managed atomic processing support failure transparency and exactly-once input processing [4]. Our tool does not place the external generator inside such a transactional runtime. It exposes uncertainty at that boundary and restricts one product entry path, rather than claiming an equivalent processing guarantee.

Temporal’s Activity documentation describes the same lost-completion boundary: a worker may finish an Activity and crash before reporting completion, leading to a retry. It recommends idempotent Activities [5]. This is an important comparison because durable workflow state alone does not eliminate uncertainty about external side effects. Our same-run guard withholds a submission under explicit uncertainty, but supplies no equivalent general workflow-execution or automatic reconciliation guarantee.

OpenTelemetry represents operations through spans and uses trace context to relate observations across a request [6]. Such correlation provides a useful representation for a distributed execution path. In this tool, the additional application-specific rule is which observations permit an execution claim: backend start and terminal events must bind to completed history, and unresolved product recovery can still veto `execution_verified`. Correlation identifiers alone are not used as execution counts.

Toxiproxy provides configurable TCP connection fault injection for testing [7]. Our research proxy uses an application-level acceptance condition to select the response to suppress, while the observer separately checks execution. This comparison does not show that Toxiproxy could not support an equivalent setup; no comparative implementation or usability

measurement was performed.

Filibuster’s project overview describes synthesizing fault-injected test variations from developer integration tests to check application error handling [8]. Our fixed case set does not provide that automatic search coverage. It concentrates on evidence interpretation at a selected submission barrier and preserves a separate backend observation channel. The comparison uses the project overview, not a reproduction of Filibuster’s research results.

This is a documentation-based comparison of responsibilities, not a performance benchmark or a feature-absence survey. The practical contribution is making the relationship between client ambiguity, backend observation and permitted claims inspectable for a local generation workflow. The comparisons delimit the tool responsibilities; they do not establish superiority, exhaustive novelty, or measured developer productivity.

VII. CONCLUSION

The tool preserves ambiguous client outcomes alongside separately bound execution evidence and implements a conservative guard within an unresolved run. The controlled Windows cases demonstrate execution visibility under lost acceptance responses, while exposing the distinction between a new successful call and reconciliation of the original run. These bounded observations support an evidence-preserving tool description. They do not establish automatic unknown-job recovery, deployment-wide reliability or natural duplicate-execution rates.

DATA, CODE AND RESPONSIBLE USE

The manuscript source includes derived Windows evidence, retained CPU-record pointers, a deterministic audit script and figure sources. Models, private configuration, generated images and raw host paths are not redistributed in this package. The analysis concerns controlled software executions; it makes no claim about human participants or image-quality acceptance. Human-subjects or institutional-ethics determinations are not inferred from this scope.

AI TOOL USAGE

OpenAI Codex assisted with drafting portions of the manuscript, source inspection, record consistency checks, and code for Figs. 1–2. The review conducted with this tool was internal and within a single context, not independent peer review. Human authors must verify the content, references, attribution, and final submission. No AI-generated data are reported as experimental observations.

REFERENCES

- [1] A. D. Birrell and B. J. Nelson, “Implementing remote procedure calls,” *ACM Transactions on Computer Systems*, vol. 2, no. 1, pp. 39–59, 1984. [Online]. Available: <https://www.cs.cmu.edu/~15712/papers/birrell84.pdf>
- [2] M. Featonby, “Making retries safe with idempotent apis,” Amazon Builders’ Library, accessed September 27, 2026. [Online]. Available: <https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/>
- [3] J. Li, “Where does exactly-once live? model, harness, and tool-contract effects on duplicate side effects in LLM agents,” arXiv preprint arXiv:2609.29095v1, 2026, version 1, September 24, 2026. [Online]. Available: <https://arxiv.org/abs/2609.29095v1>
- [4] S. Mukherjee, N. J. Raj, K. Govindraj, P. Deligiannis, C. Ravichandran, A. Lal, A. Rastogi, and R. Krishnaswamy, “Reliable state machines: A framework for programming reliable cloud services,” in *33rd European Conference on Object-Oriented Programming (ECOOP 2019)*, ser. Leibniz International Proceedings in Informatics, vol. 134, 2019, pp. 18:1–18:29. [Online]. Available: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2019.18>
- [5] Temporal Technologies, “Activity definition,” official documentation. Accessed September 27, 2026. [Online]. Available: <https://docs.temporal.io/activity-definition>
- [6] OpenTelemetry Authors, “Traces,” official documentation. Accessed September 27, 2026. [Online]. Available: <https://opentelemetry.io/docs/concepts/signals/traces/>
- [7] Shopify, “Toxiproxy,” official project documentation. Accessed September 27, 2026. [Online]. Available: <https://github.com/Shopify/toxiproxy>
- [8] Filibuster, “Reliability starts with the application,” official project overview. Accessed September 27, 2026. [Online]. Available: <https://filibuster.cloud/>